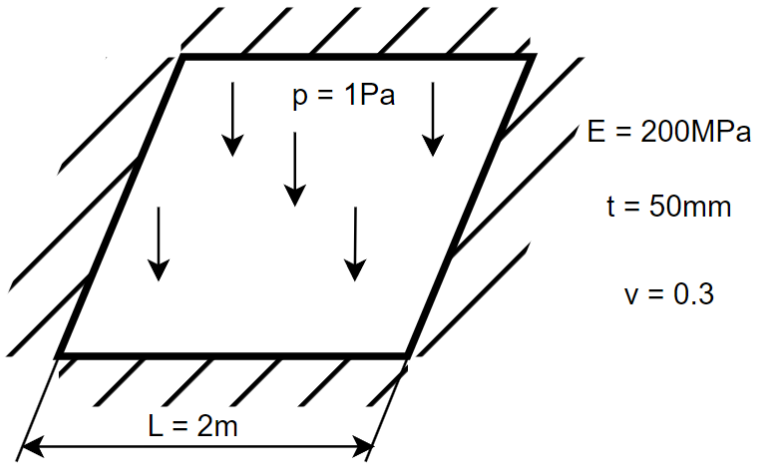
Placas

Una placa de 2m de lado y 50mm de espeso, empotrada en todos sus extremos es sujeta a una carga distribuida uniforme de 1Pa.
Considerando que la placa es de acero (E = 200MPa, v = 0.3), comparar comportamiento de

- Placas MZC
- Placas de Mindlin
- Sólidos 3D



La solución analítica (para placas de Kirchoff) en el centro de la placa está dada por:

$$w_{center} = -0{,}00126 \cdot p \cdot \left(\frac{L}{2}\right)^4 \cdot \frac{1}{D}$$

$$D = \frac{Et^3}{12(1-\nu^2)}$$

Recordar consideraciones de los elementos estudiados:

- MZC
  - Pueden representar placas delgadas ($\frac{t}{L} < 0.05$)
  - Sólo utilizable para elementos rectangulares (no isoparemetrico)
- Mindlin
  - Admiten corte transversal: Representan mejor placas gruesas ($0.05 < \frac{t}{L} < 0.1$)
  - Sufren de *shear locking* cuando $L_e \gg t$
  - Permiten formulación isoparamétrica
- Sólidos 3D
  - Tengo que representar un campo de deflexión: que elementos uso? Cuántos en el espesor? (MSC I)
  - Por qué lo uso? $\frac{t}{L} > 0.1$ Secciones dejan de permanecer planas

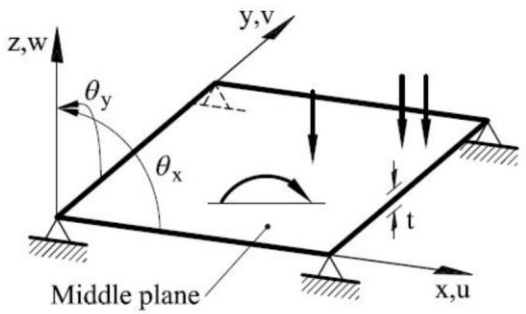
Está bien usar una solución analítica de placa de Kirchoff?

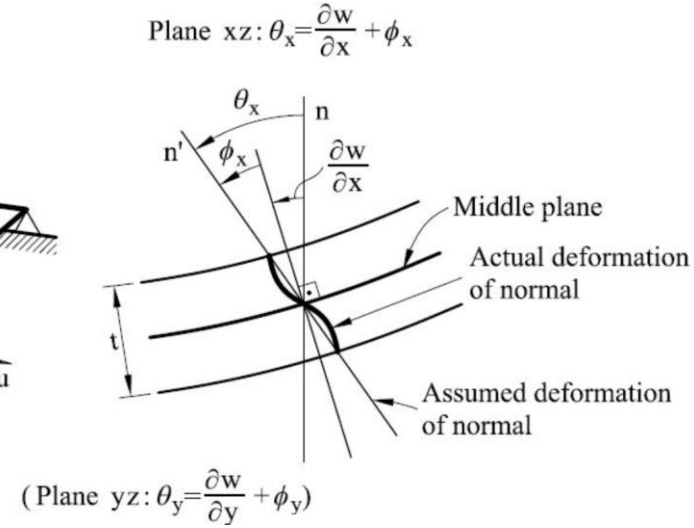
$\frac{t}{L}$ aplica sobre el problema, no sobre el largo de cada elemento al discretizar

Placas - Mindlin

- 3 DOF por nodo (w, ϕ_x, ϕ_y)
- Libertad para elegir cantidad de nodos:
  - Triangulares (3, 6)
  - Cuadriláteros (4, 8, 9, 12, 16)
- **<u>Admite formulación isoparamétrica!! (MZC no permite)</u>**

Funciones de forma 4 nodos: bilineales (1, x, y, xy) = Q4
Se extiende a resto de cuadriláteros





Desplazamientos

$$u(x,y,z) = -z\theta_x(x,y)$$
$$v(x,y,z) = -z\theta_y(x,y)$$
$$w(x,y,z) = w(x,y)$$

$$\theta_x = \frac{\partial w}{\partial x} + \phi_x$$

$$\theta_y = \frac{\partial w}{\partial y} + \phi_y$$

Deformaciones

$$\boldsymbol{\varepsilon} = \begin{Bmatrix} \varepsilon_x \\ \varepsilon_y \\ \gamma_{xy} \\ \cdots\cdots \\ \gamma_{xz} \\ \gamma_{yz} \end{Bmatrix} = \begin{Bmatrix} -z\dfrac{\partial \theta_x}{\partial x} \\ -z\dfrac{\partial \theta_y}{\partial y} \\ -z\left(\dfrac{\partial \theta_x}{\partial y} + \dfrac{\partial \theta_y}{\partial x}\right) \\ \cdots\cdots\cdots\cdots\cdots \\ \dfrac{\partial w}{\partial x} - \theta_x \\ \dfrac{\partial w}{\partial y} - \theta_y \end{Bmatrix} = \begin{Bmatrix} \boldsymbol{\varepsilon}_b \\ \cdots\cdots \\ \boldsymbol{\varepsilon}_s \end{Bmatrix}$$

Tensiones

$$\boldsymbol{\sigma}_b = \mathbf{D}_b \boldsymbol{\varepsilon}_b$$

$$\mathbf{D}_b = \frac{E}{1-\nu^2}\begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{bmatrix}$$

$$\boldsymbol{\sigma}_s = \mathbf{D}_s \boldsymbol{\varepsilon}_s$$

$$\mathbf{D}_s = \begin{bmatrix} k_{11}\bar{D}_{s_{11}} & k_{12}\bar{D}_{s_{12}} \\ k_{12}\bar{D}_{s_{12}} & k_{22}\bar{D}_{s_{22}} \end{bmatrix}$$

$$k_{11} = k_{12} = \frac{5}{6}; k_{22} = 0$$

Placas – Mindlin - Rigidez

Conviene usar esfuerzos y deformaciones generalizadas:

- Asumo la distribución de tensiones, la considero dentro del constiutivo y caracterizo

Variación con el espesor

$$\hat{\boldsymbol{\sigma}} = \begin{Bmatrix} \hat{\boldsymbol{\sigma}}_b \\ \cdots\cdots \\ \hat{\boldsymbol{\sigma}}_s \end{Bmatrix} = \begin{Bmatrix} M_x \\ M_y \\ M_{xy} \\ \cdots\cdots \\ Q_x \\ Q_y \end{Bmatrix} = \int_{-\frac{t}{2}}^{+\frac{t}{2}} \begin{Bmatrix} -z\sigma_x \\ -z\sigma_y \\ -z\tau_{xy} \\ \cdots\cdots \\ \tau_{xz} \\ \tau_{yz} \end{Bmatrix} dz = \int_{-\frac{t}{2}}^{+\frac{t}{2}} \begin{Bmatrix} -z\boldsymbol{\sigma}_b \\ \cdots\cdots \\ \boldsymbol{\sigma}_s \end{Bmatrix} dz = \int_{-\frac{t}{2}}^{+\frac{t}{2}} \mathbf{S}^T \boldsymbol{\sigma}\, dz$$

$$\hat{\boldsymbol{\varepsilon}}_b = \begin{Bmatrix} \dfrac{\partial \theta_x}{\partial x} \\ \dfrac{\partial \theta_y}{\partial y} \\ \left(\dfrac{\partial \theta_x}{\partial y} + \dfrac{\partial \theta_y}{\partial x}\right) \end{Bmatrix}$$

$$\hat{\boldsymbol{\varepsilon}}_s = \begin{Bmatrix} \dfrac{\partial w}{\partial x} - \theta_x \\ \dfrac{\partial w}{\partial y} - \theta_y \end{Bmatrix}$$

$$\begin{Bmatrix} \boldsymbol{\sigma}_b \\ \boldsymbol{\sigma}_s \end{Bmatrix} = \mathbf{D} \begin{Bmatrix} -z\hat{\boldsymbol{\varepsilon}}_b \\ \hat{\boldsymbol{\varepsilon}}_s \end{Bmatrix} \qquad \mathbf{D} = \begin{bmatrix} \mathbf{D}_b & \mathbf{0} \\ \mathbf{0} & \bar{\mathbf{D}}_s \end{bmatrix}$$

$$\hat{\boldsymbol{\sigma}} = \begin{Bmatrix} \hat{\boldsymbol{\sigma}}_b \\ \cdots \\ \hat{\boldsymbol{\sigma}}_s \end{Bmatrix} = \int_{-\frac{t}{2}}^{+\frac{t}{2}} \mathbf{S}^T \mathbf{D} \boldsymbol{\varepsilon}\, dz = \int_{-\frac{t}{2}}^{+\frac{t}{2}} \mathbf{S}^T \mathbf{D} \mathbf{S} \hat{\boldsymbol{\varepsilon}}\, dz = \int_{-\frac{t}{2}}^{+\frac{t}{2}} \begin{Bmatrix} z^2 \mathbf{D}_b \hat{\boldsymbol{\varepsilon}}_b \\ \cdots\cdots \\ \mathbf{D}_s \hat{\boldsymbol{\varepsilon}}_s \end{Bmatrix} dz = \hat{\mathbf{D}} \hat{\boldsymbol{\varepsilon}}$$

$$\hat{\mathbf{D}} = \int_{-\frac{t}{2}}^{+\frac{t}{2}} \mathbf{S}^T \mathbf{D} \mathbf{S}\, dz = \int_{-\frac{t}{2}}^{+\frac{t}{2}} \begin{bmatrix} z^2 \mathbf{D}_b & \mathbf{0} \\ \mathbf{0} & \mathbf{D}_s \end{bmatrix} dz = \begin{bmatrix} \hat{\mathbf{D}}_b & \mathbf{0} \\ \mathbf{0} & \hat{\mathbf{D}}_s \end{bmatrix}$$

$$\hat{\mathbf{D}}_b = \frac{t^3}{12} \mathbf{D}_b$$

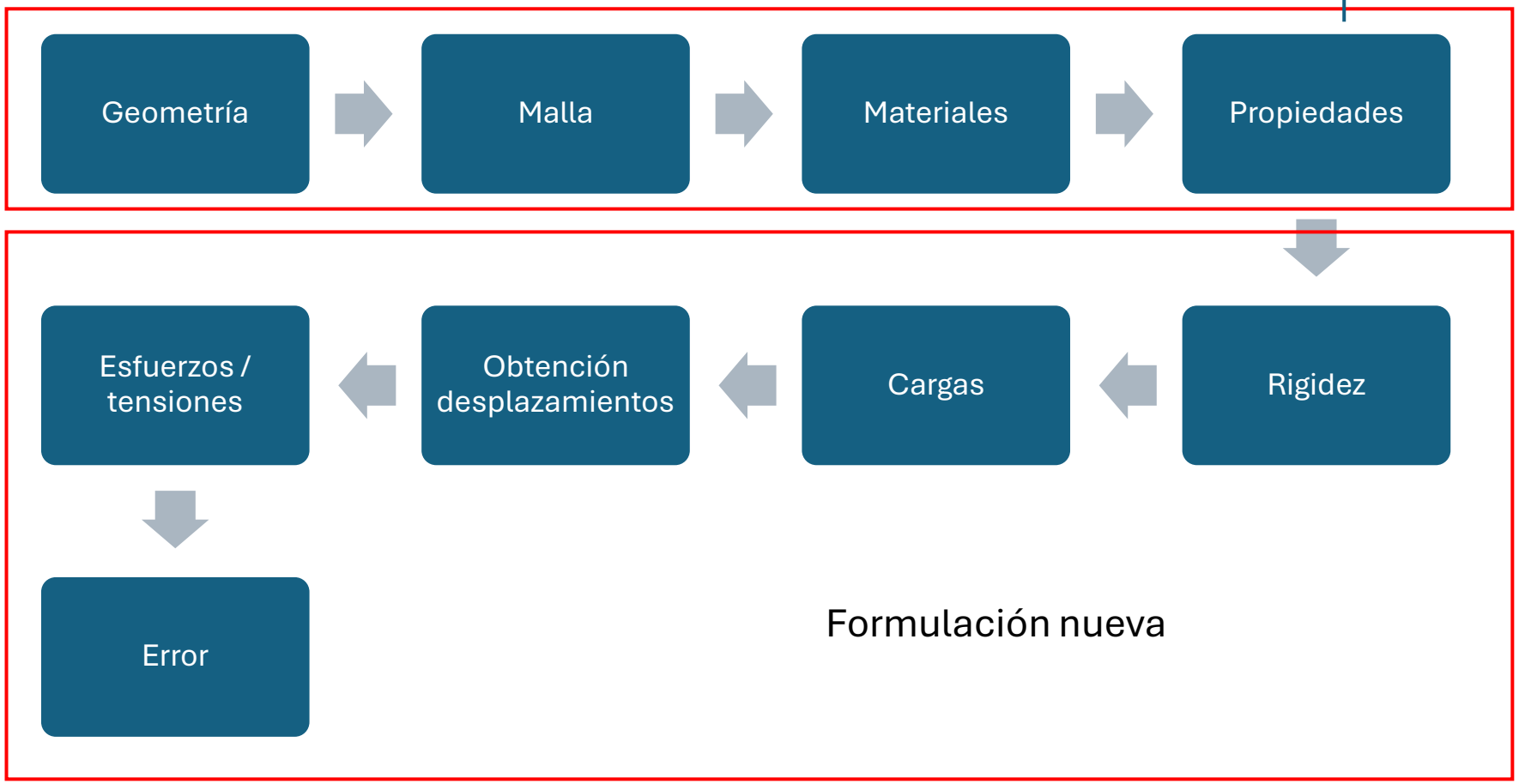
$$\hat{\mathbf{D}}_s = t\mathbf{D}_s = \frac{5}{6} t G \mathbf{I}_2$$

Integro de manera analítica en el espesor porque ya sé la distribución

Placas – Mindlin – Estructura de programa

MSC I

- Material
- Espesor




```
function [Cb, Cs] = getElementConstitutive(material, t)

E = material.E;
G = material.G;
nu = material.nu;

Ep = E/(1-nu^2);
Gp = 5/6*G;

Cb = t^3/12 * ...
[Ep nu*Ep 0
 nu*Ep Ep 0
 0 0 G];

Cs = t * ...
[ Gp 0
 0 Gp];

end
```

Ya generalizadas

Placas – Mindlin - Rigidez

$$\mathbf{K}_{ij}^{(e)} = \iint_{A^{(e)}} \mathbf{B}_i{}^T \hat{\mathbf{D}} \mathbf{B}_j dA$$

Separación operadores derivada

$$\mathbf{B}_i = \begin{Bmatrix} \mathbf{B}_{b_i} \\ \dots \\ \mathbf{B}_{s_i} \end{Bmatrix} \quad \text{with} \quad \mathbf{B}_{b_i} = \begin{bmatrix} 0 & \frac{\partial N_i}{\partial x} & 0 \\ 0 & 0 & \frac{\partial N_i}{\partial y} \\ 0 & \frac{\partial N_i}{\partial y} & \frac{\partial N_i}{\partial x} \end{bmatrix} \quad , \quad \mathbf{B}_{s_i} = \begin{bmatrix} \frac{\partial N_i}{\partial x} & -N_i & 0 \\ \frac{\partial N_i}{\partial y} & 0 & -N_i \end{bmatrix}$$

$$\frac{\partial w}{\partial x} - \phi_x \qquad \frac{\partial w}{\partial y} - \phi_y$$

$$\hat{\boldsymbol{\varepsilon}} = \begin{Bmatrix} \hat{\boldsymbol{\varepsilon}}_b \\ \dots \\ \hat{\boldsymbol{\varepsilon}}_s \end{Bmatrix} = \begin{Bmatrix} \frac{\partial \theta_x}{\partial x} \\ \frac{\partial \theta_y}{\partial y} \\ \left(\frac{\partial \theta_x}{\partial y} + \frac{\partial \theta_y}{\partial x}\right) \\ \dots\dots\dots\dots \\ \frac{\partial w}{\partial x} - \theta_x \\ \frac{\partial w}{\partial y} - \theta_y \end{Bmatrix} = \sum_{i=1}^{n} \begin{Bmatrix} \frac{\partial N_i}{\partial x}\theta_{x_i} \\ \frac{\partial N_i}{\partial y}\theta_{y_i} \\ \left(\frac{\partial N_i}{\partial y}\theta_{x_i} + \frac{\partial N_i}{\partial x}\theta_{y_i}\right) \\ \dots\dots\dots\dots\dots\dots \\ \frac{\partial N_i}{\partial x} w_i - N_i \theta_{x_i} \\ \frac{\partial N_i}{\partial y} w_i - N_i \theta_{y_i} \end{Bmatrix}$$

Integración en el área (generalización integra en el espesor)

Matriz de rigidez como suma de bending + shear

$$\mathbf{K}_{ij}^{(e)} = \iint_{A^{(e)}} [\mathbf{B}_{b_i}^T, \mathbf{B}_{s_i}^T]^T \hat{\mathbf{D}} \begin{Bmatrix} \mathbf{B}_{b_j} \\ \mathbf{B}_{s_j} \end{Bmatrix} dA = \int_{A^{(e)}} \left(\mathbf{B}_{b_i}^T \hat{\mathbf{D}}_b \mathbf{B}_{b_j} + \mathbf{B}_{s_i}^T \hat{\mathbf{D}}_s \mathbf{B}_{s_j} \right) dA = \mathbf{K}_{b_{ij}}^{(e)} + \mathbf{K}_{s_{ij}}^{(e)}$$

$$\begin{bmatrix} \hat{\mathbf{D}}_b & \mathbf{0} \\ \mathbf{0} & \hat{\mathbf{D}}_s \end{bmatrix}$$

Integración numérica por separado – permite evaluar distintas estrategias (full, selectiva, reducida)

$$\mathbf{K}_{ij}^{(e)} = \sum_{p=1}^{n_{b_p}} \sum_{q=1}^{n_{b_q}} \left[\mathbf{B}_{b_i}^T \hat{\mathbf{D}}_b \mathbf{B}_{b_j} |\mathbf{J}^e| \right]_{p,q} W_p W_q + \sum_{p=1}^{n_{s_p}} \sum_{q=1}^{n_{s_q}} \left[\mathbf{B}_{s_i}^T, \hat{\mathbf{D}}_s \mathbf{B}_{b_j} |\mathbf{J}^e| \right]_{p,q} W_p W_q$$

Placas – Mindlin - Rigidez

$$\mathbf{K}_{ij}^{(e)} = \iint_{A^{(e)}} \mathbf{B}_i{}^T \hat{\mathbf{D}} \mathbf{B}_j dA$$

```
function [Kele, KeleB, KeleS, rowIndex, columnIndex] = getElementStiffness(mesh, iEle, properties, integrationMethod)

switch integrationMethod
    case 'full'
        npgBending = 2;
        npgShear = 2;
    case 'selective'
        npgBending = 2;
        npgShear = 1;
    case 'reduced'
        npgBending = 1;
        npgShear = 1;
    otherwise
        error('Integration method %s Not supported', integrationMethod)
end

property = properties(mesh.elements.property(iEle));
Cb = property.Cb; Cs = property.Cs;
nodesEle = mesh.nodes.position(mesh.elements.connectivity(iEle,:),:);
nDofEle = numel(mesh.nodes.dof(mesh.elements.connectivity(iEle,:), :));
eleDofs = mesh.elements.dof(iEle, :)';

KeleB = zeros(nDofEle);
KeleS = zeros(nDofEle);

[wpgB,upgB,npgB] = gauss(repmat(npgBending, 2, 1));
[wpgS,upgS,npgS] = gauss(repmat(npgShear, 2, 1));

dNB = shapefunsder(upgB,'Q4');
dNS = shapefunsder(upgS,'Q4');

[NiS,~] = shapefuns(upgS,'Q4');
```

Integración numérica por separado – depende del elemento elegido (En este caso hardcodeado, podría identificarse a través de *nNodEle*)

- Cuadratura completa: 2x2
- Para integración reducida: 1x1 (1 orden menos)

$$\begin{bmatrix} \hat{\mathbf{D}}_b & \mathbf{0} \\ \mathbf{0} & \hat{\mathbf{D}}_s \end{bmatrix}$$

$$\hat{\mathbf{D}}_b = \frac{t^3}{12}\mathbf{D}_b$$

$$\hat{\mathbf{D}}_s = t\mathbf{D}_s = \frac{5}{6}tG\mathbf{I}_2$$

Cuadraturas

Pido evaluar las funciones de forma bilineales en ptos de Gauss

$$\mathbf{B}_i = \begin{Bmatrix} \mathbf{B}_{b_i} \\ \dots \\ \mathbf{B}_{s_i} \end{Bmatrix} \quad \text{with} \quad \mathbf{B}_{b_i} = \begin{bmatrix} 0 & \frac{\partial N_i}{\partial x} & 0 \\ 0 & 0 & \frac{\partial N_i}{\partial y} \\ 0 & \frac{\partial N_i}{\partial y} & \frac{\partial N_i}{\partial x} \end{bmatrix} \quad , \quad \mathbf{B}_{s_i} = \begin{bmatrix} \frac{\partial N_i}{\partial x} & -N_i & 0 \\ \frac{\partial N_i}{\partial y} & 0 & -N_i \end{bmatrix}$$

$$\mathbf{K}_{ij}^{(e)} = \sum_{p=1}^{n_{b_p}} \sum_{q=1}^{n_{b_q}} \left[\mathbf{B}_{b_i}^T \hat{\mathbf{D}}_b \mathbf{B}_{b_j} |\mathbf{J}^e| \right]_{p,q} W_p W_q + \sum_{p=1}^{n_{s_p}} \sum_{q=1}^{n_{s_q}} \left[\mathbf{B}_{s_i}^T, \hat{\mathbf{D}}_s \mathbf{B}_{b_j} |\mathbf{J}^e| \right]_{p,q} W_p W_q$$

Placas – Mindlin - Rigidez

$$\mathbf{K}_{ij}^{(e)} = \underbrace{\sum_{p=1}^{n_{b_p}} \sum_{q=1}^{n_{b_q}} \left[\mathbf{B}_{b_i}^T \hat{\mathbf{D}}_b \mathbf{B}_{b_j} |\mathbf{J}^e|\right]_{p,q} W_p W_q}_{\text{Loop 1}} + \underbrace{\sum_{p=1}^{n_{s_p}} \sum_{q=1}^{n_{s_q}} \left[\mathbf{B}_{s_i}^T, \hat{\mathbf{D}}_s \mathbf{B}_{b_j} |\mathbf{J}^e|\right]_{p,q} W_p W_q}_{\text{Loop 2}}$$

```
% Bending
for ipgB=1:npgB

    jac = dNB(:,:,ipgB)*nodesEle(:,1:2);
    dNBxy = jac\dNB(:,:,ipgB);

    Bb = zeros(3,nDofEle);

    Bb(1,2:3:end) = dNBxy(1,:);
    Bb(2,3:3:end) = dNBxy(2,:);
    Bb(3,2:3:end) = dNBxy(2,:);
    Bb(3,3:3:end) = dNBxy(1,:);

    KeleB = KeleB + Bb'*Cb*Bb*wpgB(ipgB)*det(jac);
end
```

$$(12 \times 3)(3 \times 3)(3 \times 12) = (12 \times 12)$$

$$\mathbf{B}_{b_i} = \begin{bmatrix} 0 & \frac{\partial N_i}{\partial x} & 0 \\ 0 & 0 & \frac{\partial N_i}{\partial y} \\ 0 & \frac{\partial N_i}{\partial y} & \frac{\partial N_i}{\partial x} \end{bmatrix}, \qquad \hat{\mathbf{D}}_b = \frac{t^3}{12}\mathbf{D}_b$$

Hago 2 loops para elegir distintas cuadraturas

```
% Shear
for ipgS=1:npgS

    jac = dNS(:,:,ipgS)*nodesEle(:,1:2);
    dNSxy = jac\dNS(:,:,ipgS);

    Bs = zeros(2,nDofEle);

    Bs(1,1:3:end) = dNSxy(1,:);
    Bs(2,1:3:end) = dNSxy(2,:);
    Bs(1,2:3:end) = -NiS(1,:,ipgS);
    Bs(2,3:3:end) = -NiS(1,:,ipgS);

    KeleS = KeleS + Bs'*Cs*Bs*wpgS(ipgS)*det(jac);
end
```

$$(12 \times 2)(2 \times 2)(2 \times 12) = (12 \times 12)$$

$$\mathbf{B}_{s_i} = \begin{bmatrix} \frac{\partial N_i}{\partial x} & -N_i & 0 \\ \frac{\partial N_i}{\partial y} & 0 & -N_i \end{bmatrix} \qquad \hat{\mathbf{D}}_s = t\mathbf{D}_s = \frac{5}{6} t G \mathbf{I}_2$$

K_b y K_s se calculan por separado pero están acopladas!!!

```
Kele = KeleB + KeleS;
rowIndex = repmat(eleDofs,1,size(eleDofs,1));
columnIndex = repmat(eleDofs',size(eleDofs,1),1);
```

```
function K = assembleSystem(elementStifness, rowIndex, columnIndex)

I = vertcat(rowIndex{:});
J = vertcat(columnIndex{:});
S = vertcat(elementStifness{:});

K = sparse(I,J,S);

end
```

Placas – Mindlin - Cargas

```
function [Rele, eleDofs] = applyLoadOnElement(mesh, iEle, fz, mx, my, npg)

[wpg,upg,npg] = gauss(repmat(npg, 2, 1));

[Ni, ~] = shapefuns(upg, 'Q4');
dN = shapefunsder(upg, 'Q4');

eleDofs = mesh.elements.dof(iEle,:)';
nDofEle = numel(eleDofs);
Rele = zeros(nDofEle, 1);

q = reshape([fz, mx, my]', [], 1); % Unwrap 3 column vectors

nodesEle = mesh.nodes.position(mesh.elements.connectivity(iEle,:),:);
for ipg = 1:npg
    jac = dN(:,:,ipg)*nodesEle(:, 1:2);
    N = zeros(3, nDofEle);
    N(1, 1:3:end) = Ni(:,:,ipg);
    N(2, 2:3:end) = Ni(:,:,ipg);
    N(3, 3:3:end) = Ni(:,:,ipg);
    Rele = Rele + N'*N*q*wpg(ipg)*det(jac);
end
end
```

Hardcoded ☹

Valores nodales de carga distribuida!

$$\mathbf{N}_i = \begin{bmatrix} N_i & 0 & 0 \\ 0 & N_i & 0 \\ 0 & 0 & N_i \end{bmatrix} \qquad \mathbf{f}_i^{(e)} = \iint_{A^{(e)}} \mathbf{N}_i \underbrace{[f_z, m_x, m_y]^T}_{} dA$$

Campos!

$$f_z = f_z(x,y)$$
$$m_x = m_x(x,y)$$
$$m_y = m_y(x,y)$$

```
fz = P*ones(4, 1); % Constant load
mx = zeros(4, 1); my = zeros(4, 1);
```

$$q = \begin{pmatrix} f_{z_1} \\ m_{x_1} \\ m_{y_1} \\ f_{z_2} \\ \\ \vdots \\ \\ f_{z_4} \\ m_{x_4} \\ m_{y_4} \end{pmatrix} \begin{matrix} f_1^{(iEle)} \\ \\ \\ f_2^{(iEle)} \\ \\ f_3^{(iEle)} \\ \\ f_4^{(iEle)} \\ \\ \end{matrix}$$

Aproximación de carga distribuida con funciones de forma

$$(12 \times 3)(3 \times 12)(12 \times 1) = (12 \times 1)$$

Sumo en DOF elementales

```
for iEle = 1:nEle
    Rele{iEle} = applyLoadOnElement(mesh, iEle, fz, mx, my, 2);
    R(mesh.elements.dof(iEle, :)') = R(mesh.elements.dof(iEle, :)') + Rele{iEle};
end
```

$$[f_x] = \frac{N}{m^2}$$

$$[m_x] = [m_y] = \frac{N \cdot m}{m^2}$$

Placas – Mindlin – BCs & Solve

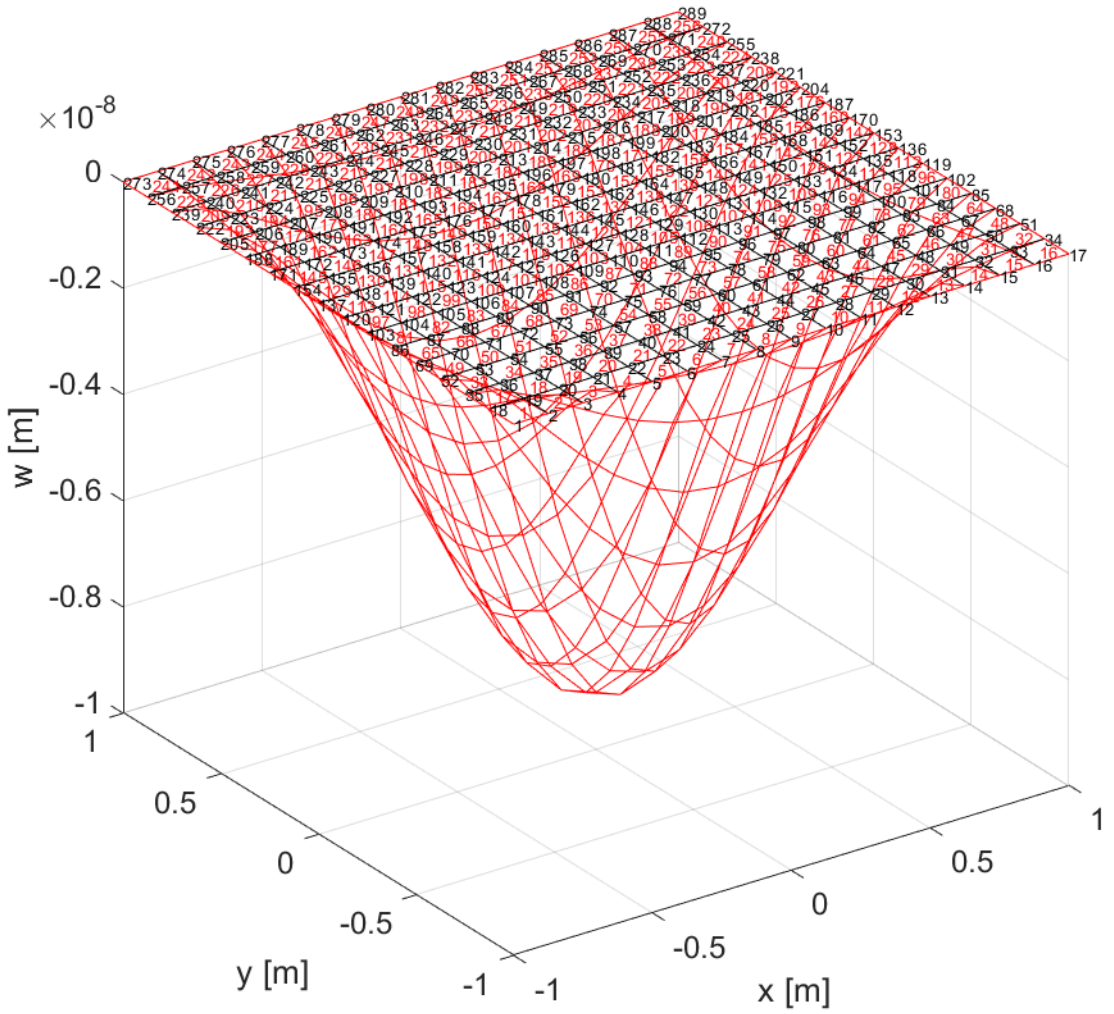
```
function bc = fixBoundary(bc, mesh, coordIdx, coordVal, dofs2fix)

boundary = abs(mesh.nodes.position(:,coordIdx)-coordVal)<1e-6;
bc(boundary,dofs2fix) = true;

end
```

```
%% BCs
bc = false(nNod, nDofNod);
bc = fixBoundary(bc, mesh, 1, -L, 1:3);
bc = fixBoundary(bc, mesh, 1, L, 1:3);
bc = fixBoundary(bc, mesh, 2, -L, 1:3);
bc = fixBoundary(bc, mesh, 2, L, 1:3);
```

$$x = \pm L$$

$$y = \pm L$$

```
function D = solveSystem(K,R,free)

R = reshape(R', [], 1);

D = zeros(size(R));

Dr = K(free,free)\R(free);

D(free) = Dr;

end
```



Proceso automatizado hasta resolver desplazamientos

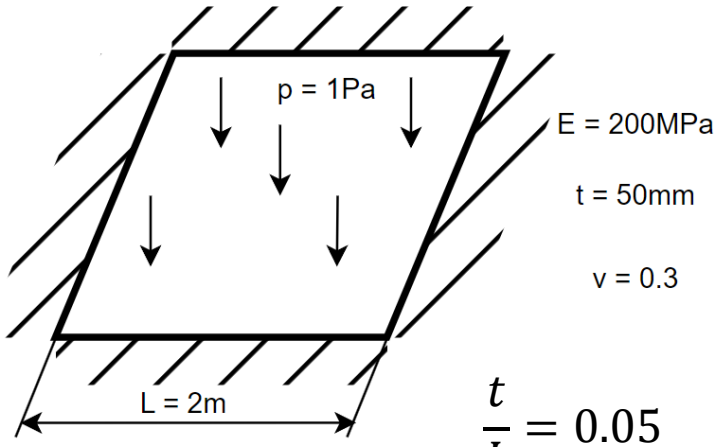
Basta generar una malla de cuadrilateros de 4 nodos que cubra el dominio (no hace falta que sea regular)

Posible mejora: admitir otras funciones de forma en cálculo de K, cálculo de R

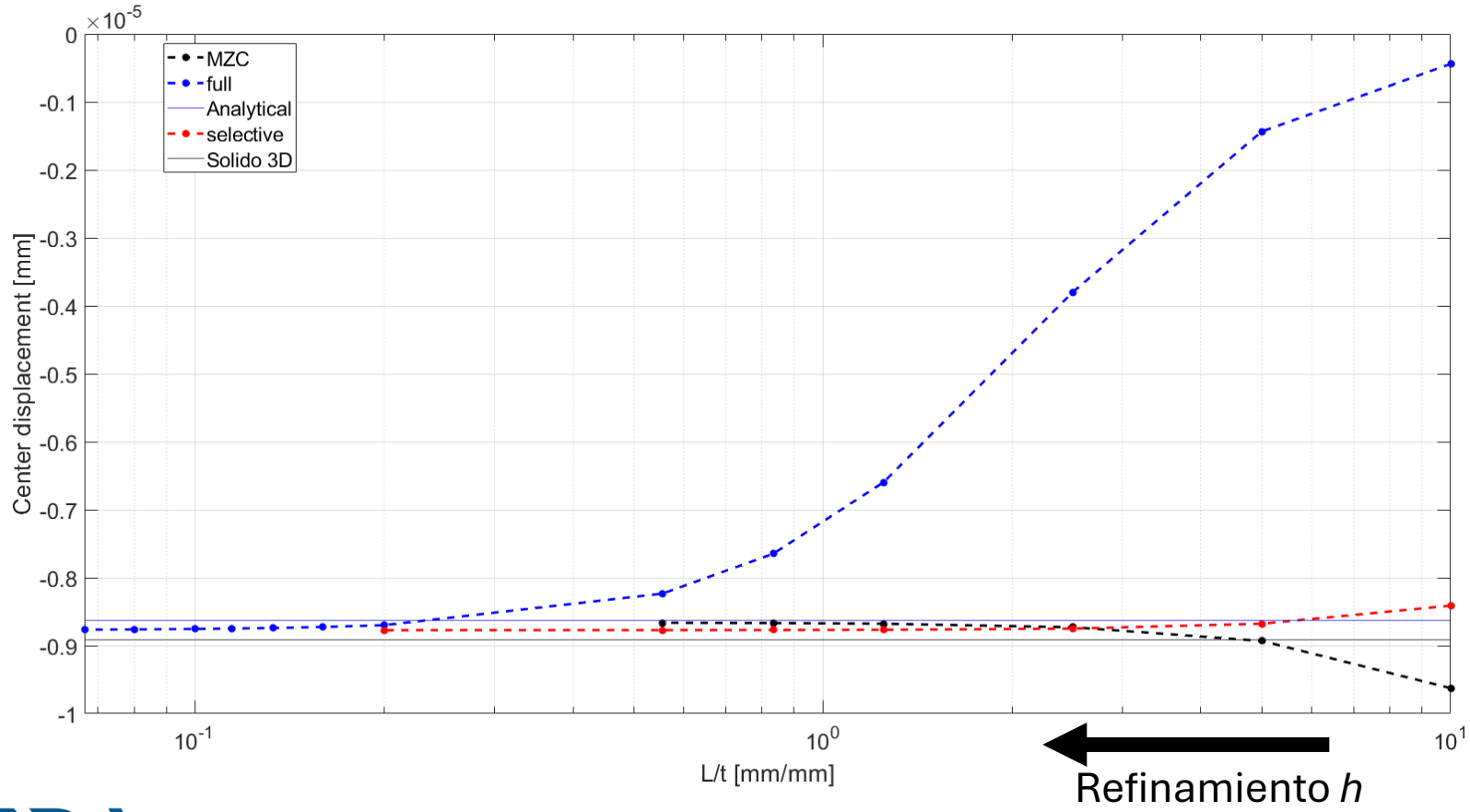
Placas – Comparación resultados

Recordar consideraciones de los elementos estudiados:

- MZC $(\frac{t}{L} < 0.05)$
- Mindlin $(0.05 < \frac{t}{L} < 0.1)$, *Shear locking* $\frac{t}{L_{element}} \ll 1$



$$\frac{t}{L} = 0.05$$



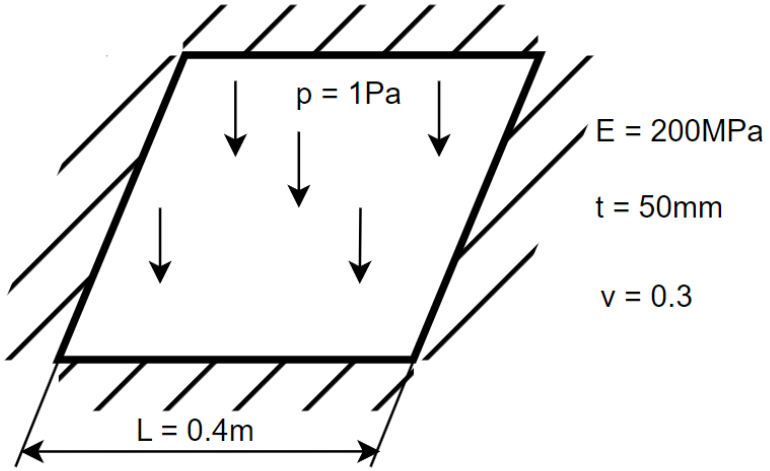
L/t = 10	Desplazamiento centro [mm]	Dif vs Analitica	Dif vs Sólido
Analytical Kirchoff	8,62E-06	N/A	-3,3%
MZC	9,62E-06	11,6%	8,0%
Mindlin Int. Full	4,40E-07	-94,9%	-95,1%
Mindlin Int. Selectiva	8,41E-06	-2,5%	-5,6%
Sólido 3-D	8,91E-06	3,4%	N/A

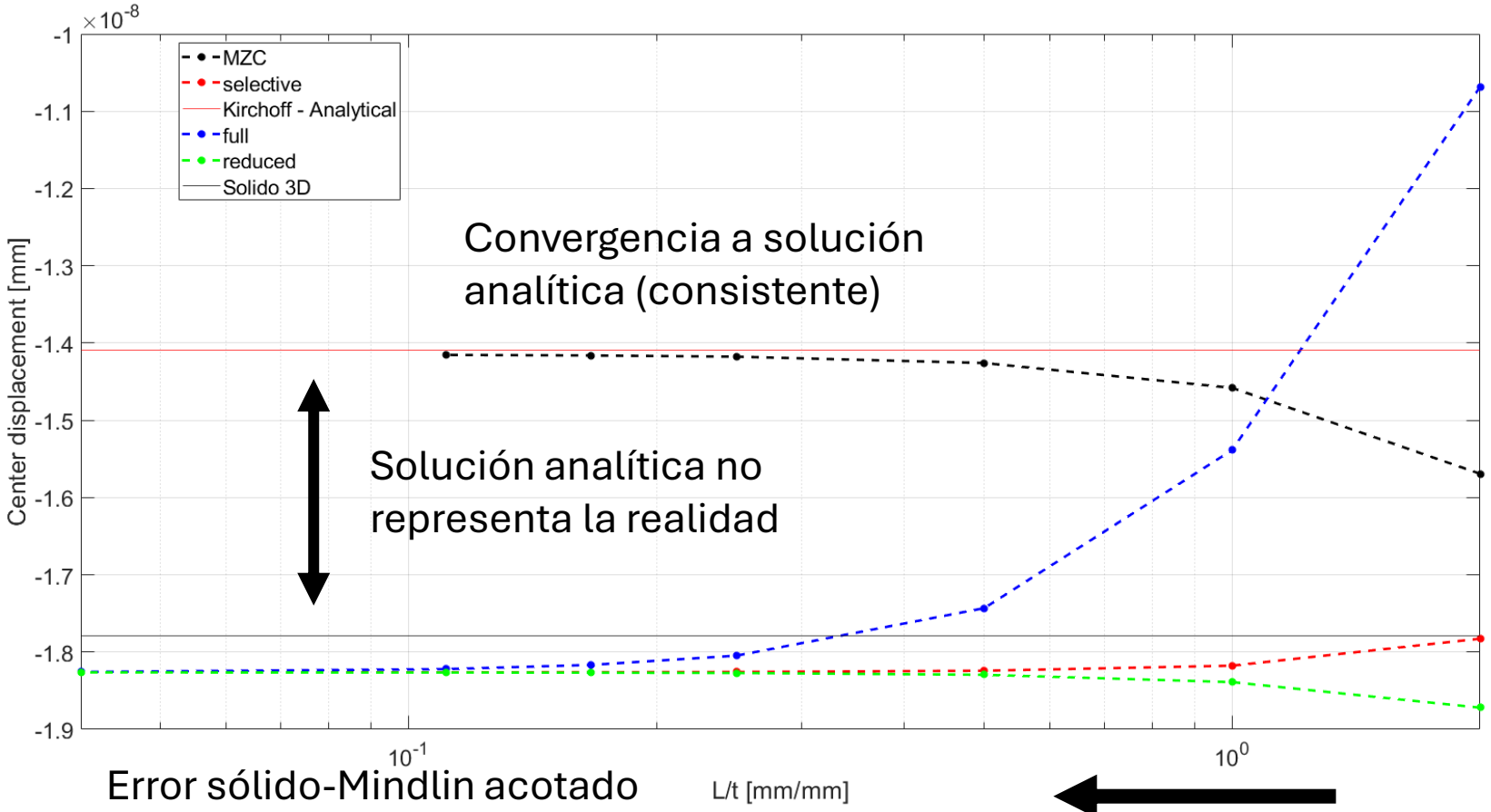
L/t mínimo	Desplazamiento centro [mm]	Dif vs Analitica	Dif vs Sólido
Analytical Kirchoff	8,62E-06	N/A	-3,3%
MZC	8,66E-06	0,5%	-2,8%
Mindlin Int. Full	8,76E-06	1,6%	-1,7%
Mindlin Int. Selectiva	8,77E-06	1,7%	-1,6%
Sólido 3-D	8,91E-06	3,4%	N/A

Placas – Comparación resultados

Cambiemos el problema.. L = 400mm, p = 1Pa, t = 50mm

$$\frac{t}{L} = 0.125 \longrightarrow$$ Fuera del rango de Kirchoff





Error sólido-Mindlin acotado $\frac{t}{L}$ en el límite de representitividad

Refinamiento *h*

L/t = 0,11	Desplazamiento centro [mm]	Dif vs Analitica	Dif vs Sólido
Analytical Kirchoff	1.41E-08	N/A	-20.8%
MZC	1.42E-08	0.5%	-20.4%
Mindlin Int. Full	1.82E-08	29.3%	2.4%
Mindlin Int. Selectiva	1.83E-08	29.6%	2.7%
Sólido 3-D	1.78E-08	26.3%	N/A

Placas – Mindlin - Esfuerzos/Tensiones

La recuperación de esfuerzos es directa una vez que tenemos los desplazamientos

$$\hat{\boldsymbol{\sigma}} = \hat{\mathbf{D}}\hat{\boldsymbol{\varepsilon}}$$

$$\hat{\boldsymbol{\sigma}} = \hat{\mathbf{D}}\mathbf{B}\mathbf{a}^{(e)} \quad \text{and} \quad \boxed{\hat{\boldsymbol{\sigma}}_b = \hat{\mathbf{D}}_b\mathbf{B}_b\mathbf{a}^{(e)}} \quad , \quad \boxed{\hat{\boldsymbol{\sigma}}_s = \hat{\mathbf{D}}_s\mathbf{B}_s\mathbf{a}^{(e)}}$$

Esfuerzos: constitutivo generalizado. 5 valores condensan todo lo que sucede en el espesor

$M_x, M_y, M_{xy}, Q_x, Q_y$

Puedo evaluar donde quiera

- Nodos
- Centro
- Puntos superconvergentes

M(i, punto, elemento)

i = 1 Mx

i = 2 My

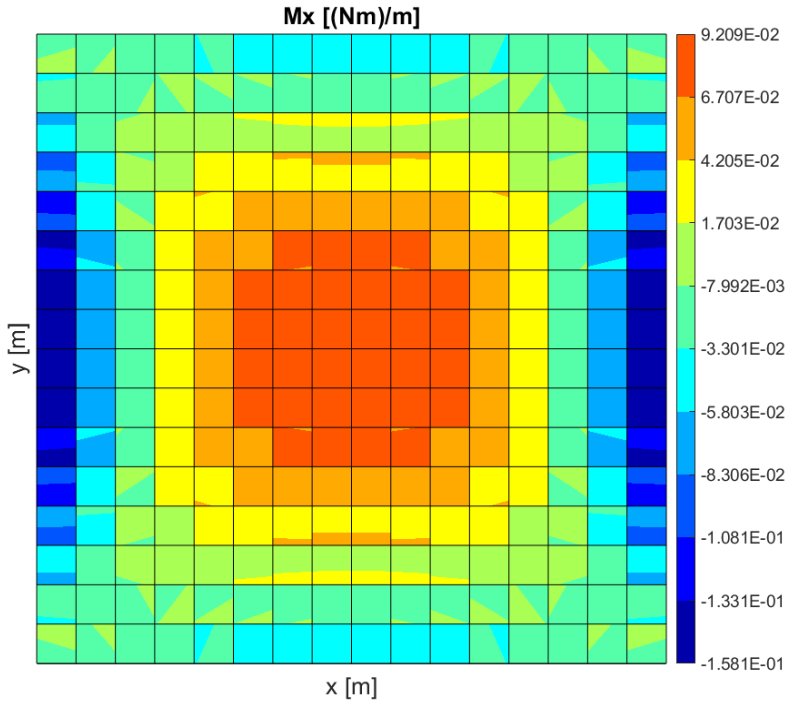
i = 3 Mxy

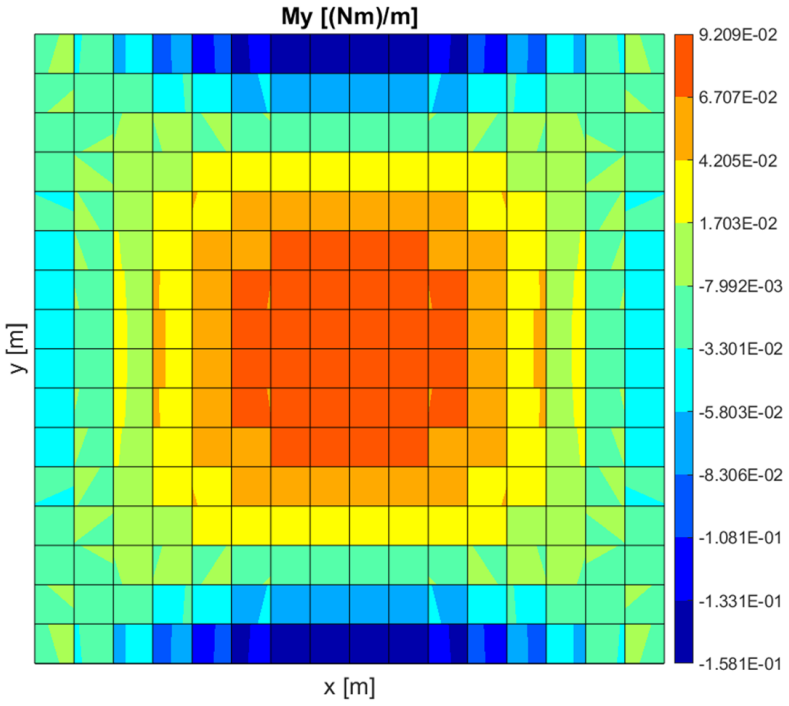
```
function [M, Q] = getCharacteristicStresses(mesh, properties, D, evaluationPoints)

nEle = size(mesh.elements.connectivity,1);

nDofNod = size(mesh.nodes.dof, 2);
nNodEle = size(mesh.elements.connectivity, 2);
nDofEle = nNodEle*nDofNod;

dNNodes = shapefunsder(evaluationPoints,'Q4');
[NiNodes,~] = shapefuns(evaluationPoints,'Q4');

nEvalPoints = length(evaluationPoints);

M = zeros(3, nEvalPoints, nEle);
Q = zeros(2, nEvalPoints, nEle);


for iEle = 1:nEle
    Cb = properties(mesh.elements.property(iEle)).Cb;
    Cs = properties(mesh.elements.property(iEle)).Cs;

    eleDofs = mesh.elements.dof(iEle,:)';
    nodesEle = mesh.nodes.position(mesh.elements.connectivity(iEle,:),:);
    De = D(eleDofs);

    for iEval = 1:nEvalPoints
        jac = dNNodes(:,:,iEval)*nodesEle(:,1:2);
        dNBxy = jac\dNNodes(:,:,iEval);
        Bb = zeros(3,nDofEle);

        Bb(1,2:3:end) = dNBxy(1,:);
        Bb(2,3:3:end) = dNBxy(2,:);
        Bb(3,2:3:end) = dNBxy(2,:);
        Bb(3,3:3:end) = dNBxy(1,:);

        Bs = zeros(2,nDofEle);

        Bs(1,1:3:end) = dNBxy(1,:);
        Bs(2,1:3:end) = dNBxy(2,:);
        Bs(1,2:3:end) = -NiNodes(1,:,iEval);
        Bs(2,3:3:end) = -NiNodes(1,:,iEval);

        M(:, iEval, iEle) = Cb*Bb*De;
        Q(:, iEval, iEle) = Cs*Bs*De;
    end
end

end
```

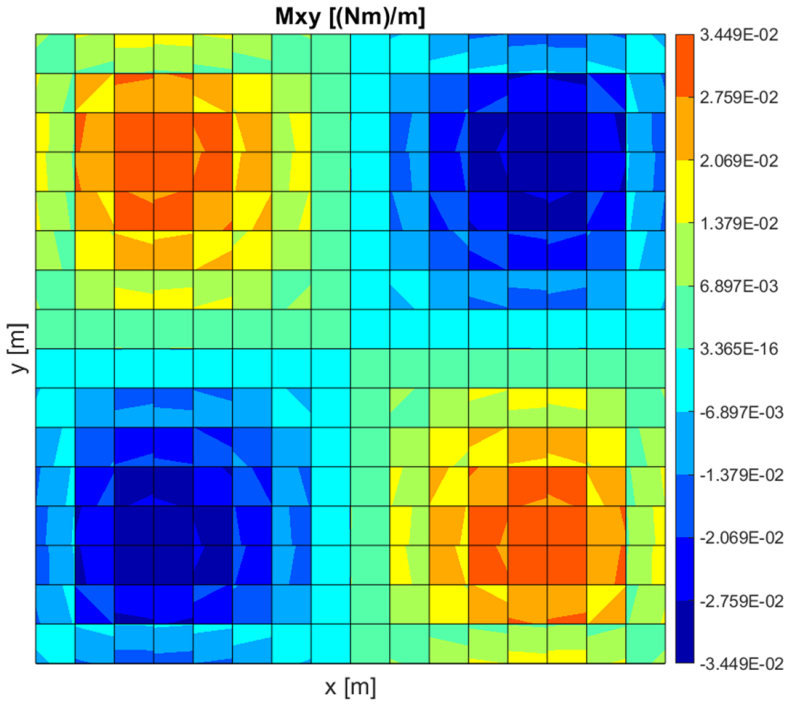
$$(3 \times 3) \cdot (3 \times 12)(12 \times 1) = (3 \times 1)$$

$$(2 \times 2) \cdot (2 \times 12)(12 \times 1) = (2 \times 1)$$

Placas – Mindlin - Tensión/Error







Por formulación $w \in C^1, \frac{\partial^2 w}{\partial x^2}, \frac{\partial^2 w}{\partial y^2}, \frac{\partial^2 w}{\partial x \partial y}$ discontinuas!

Seguimos procedimiento para cálculo de error η
Comparamos energía de deformación U vs resultado "Mejorado"

- Promedio en los nodos
- Extrapolación de puntos superconvergentes

OJO: Hay que pasar de esfuerzos a tensiones, o de deformaciones generalizadas a deformaciones

$$\left.\begin{aligned}
\|U\|^2 &= \sum_{i=1}^{m} \int_{v_e} \{\varepsilon\}_i^T [E] \{\varepsilon\}_i \, dv \\
\|e\|^2 &= \sum_{i=1}^{m} \int_{v_e} \left(\{\varepsilon^*\}_i - \{\varepsilon\}_i\right)^T [E] \left(\{\varepsilon^*\}_i - \{\varepsilon\}_i\right) dv \\
\|e\|^2 &= \sum_{i=1}^{m} \int_{v_e} \left(\{\sigma^*\}_i - \{\sigma\}_i\right)^T [E]^{-1} \left(\{\sigma^*\}_i - \{\sigma\}_i\right) dv
\end{aligned}\right\}$$

$$\eta = \sqrt{\frac{\|e\|^2}{\|e\|^2 + \|U\|^2}}$$

Placas – Mindlin - Extensión a cáscaras planas

- Agregado de esfuerzos **membranales** a formulación: Matriz B_m, elementos planos (plane stress)
- 3 DOF placas + 2 DOF membranales

$$\mathbf{K}'^{(e)}_{ij} = \iint_{A^{(e)}} \left[\mathbf{B}'^{T}_{m_i}, \mathbf{B}'^{T}_{b_i}, \mathbf{B}'^{T}_{s_i}\right] \begin{bmatrix} \mathbf{D}'_m & \hat{\mathbf{D}}'_{mb} & 0 \\ \hat{\mathbf{D}}'_{mb} & \hat{\mathbf{D}}'_b & 0 \\ 0 & 0 & \hat{\mathbf{D}}'_s \end{bmatrix} \begin{Bmatrix} \mathbf{B}'_{m_j} \\ \mathbf{B}'_{b_j} \\ \mathbf{B}'_{s_j} \end{Bmatrix} dA =$$

$$= \mathbf{K}'^{(e)}_{m_{ij}} + \mathbf{K}'^{(e)}_{b_{ij}} + \mathbf{K}'^{(e)}_{s_{ij}} + \mathbf{K}'^{(e)}_{mb_{ij}} + \left[\mathbf{K}'^{(e)}_{mb_{ij}}\right]^T$$

- Matriz local (eje primado)
- Expansión a cáscaras PLANAS con inclinación (no yacen en plano XY)

Rotación por nodo (desplazamientos y giros)

$$\mathbf{L}^{(e)}_i = \begin{bmatrix} \underset{3\times3}{\boldsymbol{\lambda}^{(e)}} & \mathbf{0} \\ \mathbf{0} & \underset{2\times3}{\hat{\boldsymbol{\lambda}}^{(e)}} \end{bmatrix} \quad , \quad \boldsymbol{\lambda}^{(e)} = \begin{bmatrix} \lambda_{xx'} & \lambda_{xx'} & \lambda_{x'z} \\ \lambda_{y'x} & \lambda_{y'y} & \lambda_{y'z} \\ \lambda_{z'x} & \lambda_{z'y} & \lambda_{z'z} \end{bmatrix}^{(e)} \quad \hat{\boldsymbol{\lambda}}^{(e)} = \begin{bmatrix} -\lambda_{y'x} & -\lambda_{y'y} & -\lambda_{y'z} \\ \lambda_{x'x} & \lambda_{x'y} & \lambda_{x'z} \end{bmatrix}^{(e)}$$

$$\mathbf{K}^{(e)} = \left[\mathbf{T}^{(e)}\right]^T \mathbf{K}'^{(e)} \mathbf{T}^{(e)} \quad ; \quad \mathbf{f}^{(e)} = [\mathbf{T}]^T \mathbf{f}'^{(e)} \qquad \underset{5n\times6n}{\mathbf{T}^{(e)}} = \begin{array}{c} \begin{array}{ccc} 1 & 2 \cdots & n \end{array} \\ \begin{bmatrix} \mathbf{L}^{(e)}_1 & & \\ & \ddots & \\ & & \mathbf{L}^{(e)}_n \end{bmatrix} \end{array} \begin{array}{c} 1 \\ 2 \\ \vdots \\ n \end{array}$$

PLANAS $\quad \mathbf{L}^{(e)}_1 = \mathbf{L}^{(e)}_2 = \ldots = \mathbf{L}^{(e)}_n.$

Recomendación bibliografía: “Structural Analysis with the Finite Element Method, Vol.2 Beams, Plates and Shells” Oñate E, Springer